Python Activity-Based Learning

Case Study 37: Nested Loop Calendar Generator

1. Problem Statement

Generate a formatted monthly calendar using a supplied starting weekday and number of days in the month.

2. Objectives

- Use nested loops to print calendar rows and columns.
- Use arithmetic and conditional statements to align dates.
- Validate the starting weekday and month length.

3. Assumptions

- Weekdays are numbered Monday=0 through Sunday=6.
- The number of days must be between 28 and 31.
- The program prints a generic month; month name and leap-year calculation are not required.

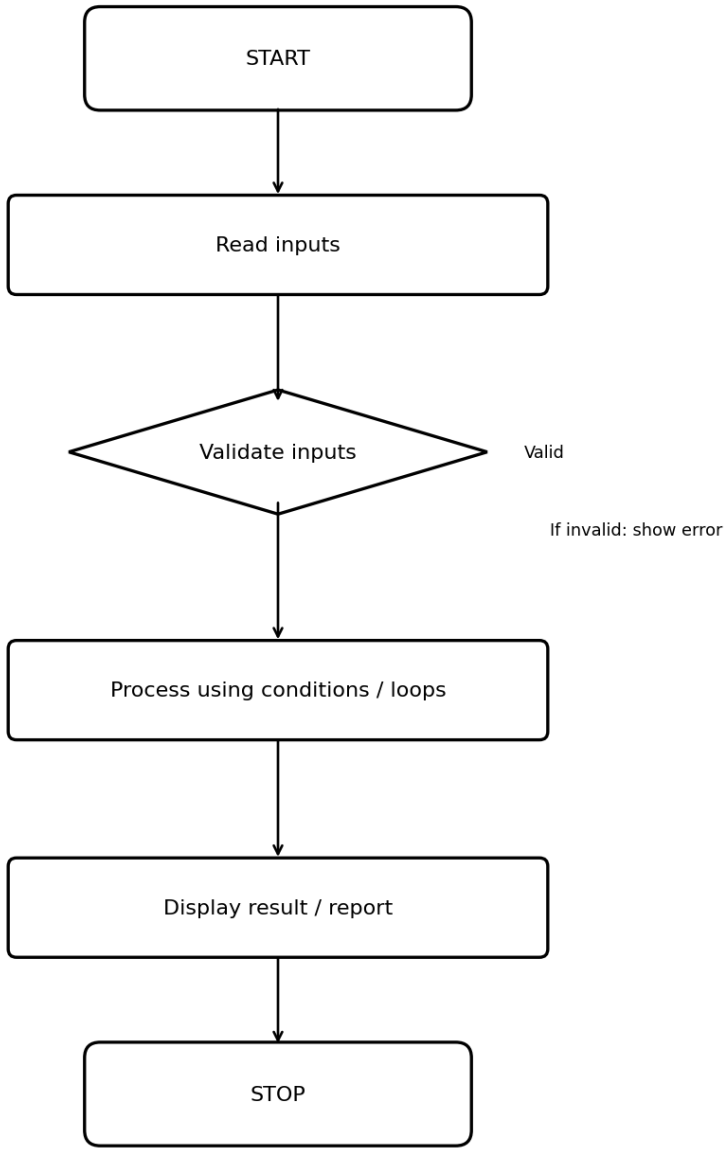
4. Algorithm

1. Read month name, starting weekday index, and number of days.
2. Validate the weekday and day count.
3. Print weekday headings.
4. Use nested loops to print blank positions before day 1 and then dates in seven columns.
5. Start a new line after each Sunday.

5. Flowchart

The diagram below shows the overall program flow. The validation decision represents the input checks described in the algorithm.

Flowchart: Nested Loop Calendar Generator



6. Pseudocode

```
START
READ month_name, start_day, days
IF start_day not in 0..6 OR days not in 28..31 THEN DISPLAY invalid input
ELSE
    DISPLAY month_name and weekday headings
    FOR blank from 1 to start_day
        DISPLAY spaces
    END FOR
    FOR day from 1 to days
        DISPLAY day in fixed-width field
        IF (start_day + day) MOD 7 = 0 THEN DISPLAY new line
    END FOR
END IF
STOP
```

7. Python Source Code

The solution uses only basic Python features from the assigned syllabus.

```
month = input("Month name: ")
start_day = int(input("Starting weekday (Mon=0 ... Sun=6): "))
days = int(input("Number of days (28 to 31): "))

if start_day < 0 or start_day > 6 or days < 28 or days > 31:
    print("Invalid weekday or number of days.")
else:
    print("\n", month)
    print("Mo Tu We Th Fr Sa Su")
    print("   " * start_day, end="")

    for day in range(1, days + 1):
        print(f"{day:2}", end=" ")
        if (start_day + day) % 7 == 0:
            print()
    if (start_day + days) % 7 != 0:
        print()
```

8. Test Cases

Test input / condition	Expected result	Purpose
October, start 3, 31	Month begins Thursday	Dates align with Thursday column
February, start 6, 28	Month begins Sunday	28 dates printed over five rows
Start weekday 7	Invalid weekday	Error message

9. Sample Output Screenshot

Sample console output for one test run:

```
C:\Users\Badri\PycharmProjects\PythonProject-1\.venv\Scripts\python.exe "C:\Users\Badri\PycharmProjects\PythonProject-1\Case Studies\case
 study - 1.py"
Month name: 2
Starting weekday (Mon=0 ... Sun=6): 2
Number of days (28 to 31): 31

 2
Mo Tu We Th Fr Sa Su
       1  2  3  4  5
 6  7  8  9 10 11 12
13 14 15 16 17 18 19
20 21 22 23 24 25 26
27 28 29 30 31
```

10. Conclusion

The calendar generator demonstrates nested-loop thinking, date positioning, formatting, and input validation.

Note: Assumed rates, thresholds, and sample values are for this educational exercise and can be changed if the instructor supplies different rules.